第 2 章

HTML 中的 JavaScript

本章内容

- 使用<script>元素
- 行内脚本与外部脚本的比较
- 文档模式对 JavaScript 有什么影响
- 确保 JavaScript 不可用时的用户体验

将 JavaScript 引入网页，首先要解决它与网页的主导语言 HTML 的关系问题。在 JavaScript 早期，网景公司的工作人员希望在将 JavaScript 引入 HTML 页面的同时，不会导致页面在其他浏览器中渲染出问题。通过反复试错和讨论，他们最终做出了一些决定，并达成了向网页中引入通用脚本能力的共识。当初他们的很多工作得到了保留，并且最终形成了 HTML 规范。

2.1 <script>元素

将 JavaScript 插入 HTML 的主要方法是使用<script>元素。这个元素是由网景公司创造出来，并最早在 Netscape Navigator 2 中实现的。后来，这个元素被正式加入到 HTML 规范。<script>元素有下列 8 个属性。

- `async`：可选。表示应该立即开始下载脚本，但不能阻止其他页面动作，比如下载资源或等待其他脚本加载。只对外部脚本文件有效。
- `charset`：可选。使用 `src` 属性指定的代码字符集。这个属性很少使用，因为大多数浏览器不在乎它的值。
- `crossorigin`：可选。配置相关请求的 CORS（跨源资源共享）设置。默认不使用 CORS。`crossorigin="anonymous"`配置文件请求不必设置凭据标志。`crossorigin="use-credentials"`设置凭据标志，意味着出站请求会包含凭据。
- `defer`：可选。表示脚本可以延迟到文档完全被解析和显示之后再执行。只对外部脚本文件有效。在 IE7 及更早的版本中，对行内脚本也可以指定这个属性。
- `integrity`：可选。允许比对接收到的资源和指定的加密签名以验证子资源完整性（SRI，Subresource Integrity）。如果接收到的资源的签名与这个属性指定的签名不匹配，则页面会报错，脚本不会执行。这个属性可以用于确保内容分发网络（CDN，Content Delivery Network）不会提供恶意内容。
- `language`：废弃。最初用于表示代码块中的脚本语言（如`"JavaScript"`、`"JavaScript 1.2"`或`"VBScript"`）。大多数浏览器都会忽略这个属性，不应该再使用它。
- `src`：可选。表示包含要执行的代码的外部文件。

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

- type：可选。代替 language，表示代码块中脚本语言的内容类型（也称 MIME 类型）。按照惯例，这个值始终都是"text/javascript"，尽管"text/javascript"和"text/ecmascript"都已经废弃了。JavaScript 文件的 MIME 类型通常是"application/x-javascript"，不过给 type 属性这个值有可能导致脚本被忽略。在非 IE 的浏览器中有效的其他值还有"application/javascript"和"application/ecmascript"。如果这个值是 module，则代码会被当成 ES6 模块，而且只有这时候代码中才能出现 import 和 export 关键字。

使用<script>的方式有两种：通过它直接在网页中嵌入 JavaScript 代码，以及通过它在网页中包含外部 JavaScript 文件。

要嵌入行内 JavaScript 代码，直接把代码放在<script>元素中就行：

```
<script>
  function sayHi() {
    console.log("Hi!");
  }
</script>
```

包含在<script>内的代码会被从上到下解释。在上面的例子中，被解释的是一个函数定义，并且该函数会被保存在解释器环境中。在<script>元素中的代码被计算完成之前，页面的其余内容不会被加载，也不会被显示。

在使用行内 JavaScript 代码时，要注意代码中不能出现字符串</script>。比如，下面的代码会导致浏览器报错：

```
<script>
  function sayScript() {
    console.log("</script>");
  }
</script>
```

浏览器解析行内脚本的方式决定了它在看到字符串</script>时，会将其当成结束的</script>标签。想避免这个问题，只需要转义字符“\”[①]即可：

```
<script>
  function sayScript() {
    console.log("<\/script>");
  }
</script>
```

这样修改之后，代码就可以被浏览器完全解释，不会导致任何错误。

要包含外部文件中的 JavaScript，就必须使用 src 属性。这个属性的值是一个 URL，指向包含 JavaScript 代码的文件，比如：

```
<script src="example.js"></script>
```

这个例子在页面中加载了一个名为 example.js 的外部文件。文件本身只需包含要放在<script>的起始及结束标签中间的 JavaScript 代码。与解释行内 JavaScript 一样，在解释外部 JavaScript 文件时，页面也会阻塞。（阻塞时间也包含下载文件的时间。）在 XHTML 文档中，可以忽略结束标签，比如：

```
<script src="example.js"/>
```

以上语法不能在 HTML 文件中使用，因为它是无效的 HTML，有些浏览器不能正常处理，比如 IE。

① 此处的转义字符指在 JavaScript 中使用反斜杠“\”来向文本字符串添加特殊字符。——编者注

> **注意** 按照惯例，外部 JavaScript 文件的扩展名是.js。这不是必需的，因为浏览器不会检查所包含 JavaScript 文件的扩展名。这就为使用服务器端脚本语言动态生成 JavaScript 代码，或者在浏览器中将 JavaScript 扩展语言（如 TypeScript，或 React 的 JSX）转译为 JavaScript 提供了可能性。不过要注意，服务器经常会根据文件扩展来确定响应的正确 MIME 类型。如果不打算使用.js 扩展名，一定要确保服务器能返回正确的 MIME 类型。

另外，使用了 src 属性的<script>元素不应该再在<script>和</script>标签中再包含其他 JavaScript 代码。如果两者都提供的话，则浏览器只会下载并执行脚本文件，从而忽略行内代码。

<script>元素的一个最为强大、同时也备受争议的特性是，它可以包含来自外部域的 JavaScript 文件。跟<img>元素很像，<script>元素的 src 属性可以是一个完整的 URL，而且这个 URL 指向的资源可以跟包含它的 HTML 页面不在同一个域中，比如这个例子：

```
<script src="http://www.somewhere.com/afile.js"></script>
```

浏览器在解析这个资源时，会向 src 属性指定的路径发送一个 GET 请求，以取得相应资源，假定是一个 JavaScript 文件。这个初始的请求不受浏览器同源策略限制，但返回并被执行的 JavaScript 则受限制。当然，这个请求仍然受父页面 HTTP/HTTPS 协议的限制。

来自外部域的代码会被当成加载它的页面的一部分来加载和解释。这个能力可以让我们通过不同的域分发 JavaScript。不过，引用了放在别人服务器上的 JavaScript 文件时要格外小心，因为恶意的程序员随时可能替换这个文件。在包含外部域的 JavaScript 文件时，要确保该域是自己所有的，或者该域是一个可信的来源。<script>标签的 integrity 属性是防范这种问题的一个武器，但这个属性也不是所有浏览器都支持。

不管包含的是什么代码，浏览器都会按照<script>在页面中出现的顺序依次解释它们，前提是它们没有使用 defer 和 async 属性。第二个<script>元素的代码必须在第一个<script>元素的代码解释完毕才能开始解释，第三个则必须等第二个解释完，以此类推。

2.1.1 标签位置

过去，所有<script>元素都被放在页面的<head>标签内，如下面的例子所示：

```
<!DOCTYPE html>
<html>
  <head>
  <title>Example HTML Page</title>
  <script src="example1.js"></script>
  <script src="example2.js"></script>
  </head>
  <body>
  <!-- 这里是页面内容 -->
  </body>
</html>
```

这种做法的主要目的是把外部的 CSS 和 JavaScript 文件都集中放到一起。不过，把所有 JavaScript 文件都放在<head>里，也就意味着必须把所有 JavaScript 代码都下载、解析和解释完成后，才能开始渲染页面（页面在浏览器解析到<body>的起始标签时开始渲染）。对于需要很多 JavaScript 的页面，这会导致页面渲染的明显延迟，在此期间浏览器窗口完全空白。为解决这个问题，现代 Web 应用程序通常将所有 JavaScript 引用放在<body>元素中的页面内容后面，如下面的例子所示：

```
<!DOCTYPE html>
<html>
  <head>
  <title>Example HTML Page</title>
  </head>
  <body>
  <!-- 这里是页面内容 -->
  <script src="example1.js"></script>
  <script src="example2.js"></script>
  </body>
</html>
```

这样一来，页面会在处理 JavaScript 代码之前完全渲染页面。用户会感觉页面加载更快了，因为浏览器显示空白页面的时间短了。

2.1.2　推迟执行脚本

HTML 4.01 为<script>元素定义了一个叫 defer 的属性。这个属性表示脚本在执行的时候不会改变页面的结构。也就是说，脚本会被延迟到整个页面都解析完毕后再运行。因此，在<script>元素上设置 defer 属性，相当于告诉浏览器立即下载，但延迟执行。

```
<!DOCTYPE html>
<html>
  <head>
  <title>Example HTML Page</title>
  <script defer src="example1.js"></script>
  <script defer src="example2.js"></script>
  </head>
  <body>
  <!-- 这里是页面内容 -->
  </body>
</html>
```

虽然这个例子中的<script>元素包含在页面的<head>中，但它们会在浏览器解析到结束的</html>标签后才会执行。HTML5 规范要求脚本应该按照它们出现的顺序执行，因此第一个推迟的脚本会在第二个推迟的脚本之前执行，而且两者都会在 DOMContentLoaded 事件之前执行（关于事件，请参考第 17 章）。不过在实际当中，推迟执行的脚本不一定总会按顺序执行或者在 DOMContentLoaded 事件之前执行，因此最好只包含一个这样的脚本。

如前所述，defer 属性只对外部脚本文件才有效。这是 HTML5 中明确规定的，因此支持 HTML5 的浏览器会忽略行内脚本的 defer 属性。IE4~7 展示出的都是旧的行为，IE8 及更高版本则支持 HTML5 定义的行为。

对 defer 属性的支持是从 IE4、Firefox 3.5、Safari 5 和 Chrome 7 开始的。其他所有浏览器则会忽略这个属性，按照通常的做法来处理脚本。考虑到这一点，还是把要推迟执行的脚本放在页面底部比较好。

> **注意**　对于 XHTML 文档，指定 defer 属性时应该写成 defer="defer"。

2.1.3　异步执行脚本

HTML5 为<script>元素定义了 async 属性。从改变脚本处理方式上看，async 属性与 defer 类

似。当然，它们两者也都只适用于外部脚本，都会告诉浏览器立即开始下载。不过，与 defer 不同的是，标记为 async 的脚本并不保证能按照它们出现的次序执行，比如：

```
<!DOCTYPE html>
<html>
  <head>
  <title>Example HTML Page</title>
  <script async src="example1.js"></script>
  <script async src="example2.js"></script>
  </head>
  <body>
  <!-- 这里是页面内容 -->
  </body>
</html>
```

在这个例子中，第二个脚本可能先于第一个脚本执行。因此，重点在于它们之间没有依赖关系。给脚本添加 async 属性的目的是告诉浏览器，不必等脚本下载和执行完后再加载页面，同样也不必等到该异步脚本下载和执行后再加载其他脚本。正因为如此，异步脚本不应该在加载期间修改 DOM。

异步脚本保证会在页面的 load 事件前执行，但可能会在 DOMContentLoaded（参见第 17 章）之前或之后。Firefox 3.6、Safari 5 和 Chrome 7 支持异步脚本。使用 async 也会告诉页面你不会使用 document.write，不过好的 Web 开发实践根本就不推荐使用这个方法。

> **注意** 对于 XHTML 文档，指定 async 属性时应该写成 async="async"。

2.1.4 动态加载脚本

除了<script>标签，还有其他方式可以加载脚本。因为 JavaScript 可以使用 DOM API，所以通过向 DOM 中动态添加 script 元素同样可以加载指定的脚本。只要创建一个 script 元素并将其添加到 DOM 即可。

```
let script = document.createElement('script');
script.src = 'gibberish.js';
document.head.appendChild(script);
```

当然，在把 HTMLElement 元素添加到 DOM 且执行到这段代码之前不会发送请求。默认情况下，以这种方式创建的<script>元素是以异步方式加载的，相当于添加了 async 属性。不过这样做可能会有问题，因为所有浏览器都支持 createElement()方法，但不是所有浏览器都支持 async 属性。因此，如果要统一动态脚本的加载行为，可以明确将其设置为同步加载：

```
let script = document.createElement('script');
script.src = 'gibberish.js';
script.async = false;
document.head.appendChild(script);
```

以这种方式获取的资源对浏览器预加载器是不可见的。这会严重影响它们在资源获取队列中的优先级。根据应用程序的工作方式以及怎么使用，这种方式可能会严重影响性能。要想让预加载器知道这些动态请求文件的存在，可以在文档头部显式声明它们：

```
<link rel="preload" href="gibberish.js">
```

2.1.5　XHTML 中的变化

可扩展超文本标记语言（XHTML，Extensible HyperText Markup Language）是将 HTML 作为 XML 的应用重新包装的结果。与 HTML 不同，在 XHTML 中使用 JavaScript 必须指定 `type` 属性且值为 `text/javascript`，HTML 中则可以没有这个属性。XHTML 虽然已经退出历史舞台，但实践中偶尔可能也会遇到遗留代码，为此本节稍作介绍。

在 XHTML 中编写代码的规则比 HTML 中严格，这会影响使用`<script>`元素嵌入 JavaScript 代码。下面的代码块虽然在 HTML 中有效，但在 XHML 中是无效的。

```
<script type="text/javascript">
  function compare(a, b) {
    if (a < b) {
      console.log("A is less than B");
    } else if (a > b) {
      console.log("A is greater than B");
    } else {
      console.log("A is equal to B");
    }
  }
</script>
```

在 HTML 中，解析`<script>`元素会应用特殊规则。XHTML 中则没有这些规则。这意味着 `a < b` 语句中的小于号（`<`）会被解释成一个标签的开始，并且由于作为标签开始的小于号后面不能有空格，这会导致语法错误。

避免 XHTML 中这种语法错误的方法有两种。第一种是把所有小于号（`<`）都替换成对应的 HTML 实体形式（`<`）。结果代码就是这样的：

```
<script type="text/javascript">
  function compare(a, b) {
    if (a &lt; b) {
      console.log("A is less than B");
    } else if (a > b) {
      console.log("A is greater than B");
    } else {
      console.log("A is equal to B");
    }
  }
</script>
```

这样代码就可以在 XHTML 页面中运行了。不过，缺点是会影响阅读。好在还有另一种方法。

第二种方法是把所有代码都包含到一个 CDATA 块中。在 XHTML（及 XML）中，CDATA 块表示文档中可以包含任意文本的区块，其内容不作为标签来解析，因此可以在其中包含任意字符，包括小于号，并且不会引发语法错误。使用 CDATA 的格式如下：

```
<script type="text/javascript"><![CDATA[
  function compare(a, b) {
    if (a < b) {
      console.log("A is less than B");
    } else if (a > b) {
      console.log("A is greater than B");
    } else {
      console.log("A is equal to B");
    }
  }
]]></script>
```

在兼容 XHTML 的浏览器中，这样能解决问题。但在不支持 CDATA 块的非 XHTML 兼容浏览器中则不行。为此，CDATA 标记必须使用 JavaScript 注释来抵消：

```
<script type="text/javascript">
//<![CDATA[
  function compare(a, b) {
    if (a < b) {
      console.log("A is less than B");
    } else if (a > b) {
      console.log("A is greater than B");
    } else {
      console.log("A is equal to B");
    }
  }
//]]>
</script>
```

这种格式适用于所有现代浏览器。虽然有点黑科技的味道，但它可以通过 XHTML 验证，而且对 XHTML 之前的浏览器也能优雅地降级。

> **注意** XHTML 模式会在页面的 MIME 类型被指定为"application/xhtml+xml"时触发。并不是所有浏览器都支持以这种方式送达的 XHTML。

2.1.6 废弃的语法

自 1995 年 Netscape 2 发布以来，所有浏览器都将 JavaScript 作为默认的编程语言。type 属性使用一个 MIME 类型字符串来标识<script>的内容，但 MIME 类型并没有跨浏览器标准化。即使浏览器默认使用 JavaScript，在某些情况下某个无效或无法识别的 MIME 类型也可能导致浏览器跳过（不执行）相关代码。因此，除非你使用 XHTML 或<script>标签要求或包含非 JavaScript 代码，最佳做法是不指定 type 属性。

在最初采用 script 元素时，它标志着开始走向与传统 HTML 解析不同的流程。对这个元素需要应用特殊的解析规则，而这在不支持 JavaScript 的浏览器（特别是 Mosaic）中会导致问题。不支持的浏览器会把<script>元素的内容输出到页面上，从而破坏页面的外观。

Netscape 联合 Mosaic 拿出了一个解决方案，对不支持 JavaScript 的浏览器隐藏嵌入的 JavaScript 代码。最终方案是把脚本代码包含在一个 HTML 注释中，像这样：

```
<script><!--
  function sayHi(){
    console.log("Hi!");
  }
//--></script>
```

使用这种格式，Mosaic 等浏览器就可以忽略<script>标签中的内容，而支持 JavaScript 的浏览器则必须识别这种模式，将其中的内容作为 JavaScript 来解析。

虽然这种格式仍然可以被所有浏览器识别和解析，但已经不再必要，而且不应该再使用了。在 XHTML 模式下，这种格式也会导致脚本被忽略，因为代码处于有效的 XML 注释当中。

2.2　行内代码与外部文件

虽然可以直接在 HTML 文件中嵌入 JavaScript 代码，但通常认为最佳实践是尽可能将 JavaScript 代码放在外部文件中。不过这个最佳实践并不是明确的强制性规则。推荐使用外部文件的理由如下。

- **可维护性**。JavaScript 代码如果分散到很多 HTML 页面，会导致维护困难。而用一个目录保存所有 JavaScript 文件，则更容易维护，这样开发者就可以独立于使用它们的 HTML 页面来编辑代码。
- **缓存**。浏览器会根据特定的设置缓存所有外部链接的 JavaScript 文件，这意味着如果两个页面都用到同一个文件，则该文件只需下载一次。这最终意味着页面加载更快。
- **适应未来**。通过把 JavaScript 放到外部文件中，就不必考虑用 XHTML 或前面提到的注释黑科技。包含外部 JavaScript 文件的语法在 HTML 和 XHTML 中是一样的。

在配置浏览器请求外部文件时，要重点考虑的一点是它们会占用多少带宽。在 SPDY/HTTP2 中，预请求的消耗已显著降低，以轻量、独立 JavaScript 组件形式向客户端送达脚本更具优势。

比如，第一个页面包含如下脚本：

```
<script src="mainA.js"></script>
<script src="component1.js"></script>
<script src="component2.js"></script>
<script src="component3.js"></script>
...
```

后续页面可能包含如下脚本：

```
<script src="mainB.js"></script>
<script src="component3.js"></script>
<script src="component4.js"></script>
<script src="component5.js"></script>
...
```

在初次请求时，如果浏览器支持 SPDY/HTTP2，就可以从同一个地方取得一批文件，并将它们逐个放到浏览器缓存中。从浏览器角度看，通过 SPDY/HTTP2 获取所有这些独立的资源与获取一个大 JavaScript 文件的延迟差不多。

在第二个页面请求时，由于你已经把应用程序切割成了轻量可缓存的文件，第二个页面也依赖的某些组件此时已经存在于浏览器缓存中了。

当然，这里假设浏览器支持 SPDY/HTTP2，只有比较新的浏览器才满足。如果你还想支持那些比较老的浏览器，可能还是用一个大文件更合适。

2.3　文档模式

IE5.5 发明了文档模式的概念，即可以使用 `doctype` 切换文档模式。最初的文档模式有两种：**混杂模式**（quirks mode）和**标准模式**（standards mode）。前者让 IE 像 IE5 一样（支持一些非标准的特性），后者让 IE 具有兼容标准的行为。虽然这两种模式的主要区别只体现在通过 CSS 渲染的内容方面，但对 JavaScript 也有一些关联影响，或称为副作用。本书会经常提到这些副作用。

IE 初次支持文档模式切换以后，其他浏览器也跟着实现了。随着浏览器的普遍实现，又出现了第三种文档模式：**准标准模式**（almost standards mode）。这种模式下的浏览器支持很多标准的特性，但是没有标准规定得那么严格。主要区别在于如何对待图片元素周围的空白（在表格中使用图片时最明显）。

混杂模式在所有浏览器中都以省略文档开头的 doctype 声明作为开关。这种约定并不合理，因为混杂模式在不同浏览器中的差异非常大，不使用黑科技基本上就没有浏览器一致性可言。

标准模式通过下列几种文档类型声明开启：

```
<!-- HTML 4.01 Strict -->
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">

<!-- XHTML 1.0 Strict -->
<!DOCTYPE html PUBLIC
"-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<!-- HTML5 -->
<!DOCTYPE html>
```

准标准模式通过过渡性文档类型（Transitional）和框架集文档类型（Frameset）来触发：

```
<!-- HTML 4.01 Transitional -->
<!DOCTYPE HTML PUBLIC
"-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<!-- HTML 4.01 Frameset -->
<!DOCTYPE HTML PUBLIC
"-//W3C//DTD HTML 4.01 Frameset//EN"
"http://www.w3.org/TR/html4/frameset.dtd">

<!-- XHTML 1.0 Transitional -->
<!DOCTYPE html PUBLIC
"-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<!-- XHTML 1.0 Frameset -->
<!DOCTYPE html PUBLIC
"-//W3C//DTD XHTML 1.0 Frameset//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">
```

准标准模式与标准模式非常接近，很少需要区分。人们在说到“标准模式”时，可能指其中任何一个。而对文档模式的检测（本书后面会讨论）也不会区分它们。本书后面所说的**标准模式**，指的就是除混杂模式以外的模式。

2.4 <noscript>元素

针对早期浏览器不支持 JavaScript 的问题，需要一个页面优雅降级的处理方案。最终，<noscript>元素出现，被用于给不支持 JavaScript 的浏览器提供替代内容。虽然如今的浏览器已经 100%支持 JavaScript，但对于禁用 JavaScript 的浏览器来说，这个元素仍然有它的用处。

<noscript>元素可以包含任何可以出现在<body>中的 HTML 元素，<script>除外。在下列两种情况下，浏览器将显示包含在<noscript>中的内容：

- 浏览器不支持脚本；
- 浏览器对脚本的支持被关闭。

任何一个条件被满足，包含在<noscript>中的内容就会被渲染。否则，浏览器不会渲染<noscript>中的内容。

下面是一个例子：

```
<!DOCTYPE html>
<html>
  <head>
  <title>Example HTML Page</title>
  <script defer="defer" src="example1.js"></script>
  <script defer="defer" src="example2.js"></script>
  </head>
  <body>
  <noscript>
    <p>This page requires a JavaScript-enabled browser.</p>
  </noscript>
  </body>
</html>
```

这个例子是在脚本不可用时让浏览器显示一段话。如果浏览器支持脚本，则用户永远不会看到它。

2.5　小结

JavaScript 是通过`<script>`元素插入到 HTML 页面中的。这个元素可用于把 JavaScript 代码嵌入到 HTML 页面中，跟其他标记混合在一起，也可用于引入保存在外部文件中的 JavaScript。本章的重点可以总结如下。

- 要包含外部 JavaScript 文件，必须将 `src` 属性设置为要包含文件的 URL。文件可以跟网页在同一台服务器上，也可以位于完全不同的域。
- 所有`<script>`元素会依照它们在网页中出现的次序被解释。在不使用 `defer` 和 `async` 属性的情况下，包含在`<script>`元素中的代码必须严格按次序解释。
- 对不推迟执行的脚本，浏览器必须解释完位于`<script>`元素中的代码，然后才能继续渲染页面的剩余部分。为此，通常应该把`<script>`元素放到页面末尾，介于主内容之后及`</body>`标签之前。
- 可以使用 `defer` 属性把脚本推迟到文档渲染完毕后再执行。推迟的脚本原则上按照它们被列出的次序执行。
- 可以使用 `async` 属性表示脚本不需要等待其他脚本，同时也不阻塞文档渲染，即异步加载。异步脚本不能保证按照它们在页面中出现的次序执行。
- 通过使用`<noscript>`元素，可以指定在浏览器不支持脚本时显示的内容。如果浏览器支持并启用脚本，则`<noscript>`元素中的任何内容都不会被渲染。

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权